

Reinforcement Learning: Introdução

Machine Learning 2 2026.1

Francisco Ganacim, Daniel Yukimura

IMPA

June 5, 2026

Roteiro da Aula

1. **Introdução:** agente, ambiente e exemplos
2. **Multi-armed Bandits:** exploração vs. exploração
3. **Formalização:** episódios, política e a Propriedade de Markov
4. **Funções de Valor e Equações de Bellman**
5. **Métodos Elementares:** Programação Dinâmica, Monte Carlo e TD
6. **Deep Reinforcement Learning:** DQN
7. **Policy Gradient:** REINFORCE, Actor-Critic, TRPO e PPO

Introdução

O que é Reinforcement Learning?

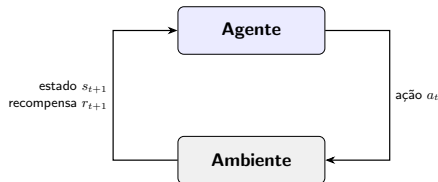
Paradigmas de aprendizado de máquina:

- ▶ **Supervisionado**: aprende de pares (*entrada, rótulo*) fornecidos.
- ▶ **Não supervisionado**: encontra estrutura em dados *sem rótulo*.
- ▶ **por Reforço (RL)**: um *Agente* aprende por **tentativa e erro**, interagindo com o *Ambiente* para maximizar *Recompensas*.

A essência do RL

- ▶ **Sem rótulos**: apenas um sinal **escalar** de recompensa, em geral esparsa e **atrasado**.
- ▶ **Decisões sequenciais**: cada ação afeta o estado futuro — e os próprios dados que o agente verá.
- ▶ **Objetivo**: maximizar a recompensa **acumulada**, não acertar uma saída isolada.

Agente e Ambiente

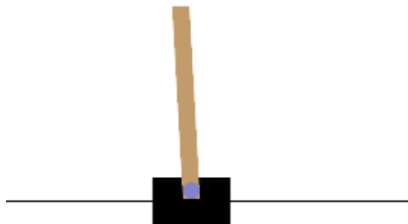


A cada passo t :

- ▶ o **Agente** observa o estado s_t e escolhe uma **ação** a_t ;
- ▶ o **Ambiente** responde com uma **recompensa** r_{t+1} e um novo **estado** s_{t+1} .

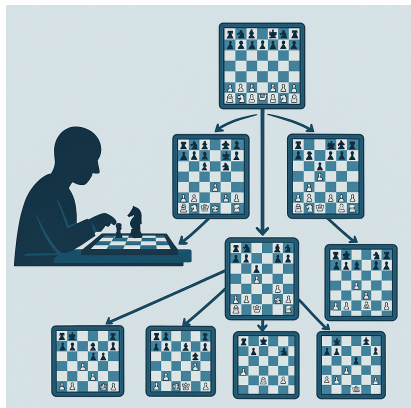
O objetivo do Agente é maximizar a **recompensa total** acumulada.

Exemplos: Cart Pole



- ▶ **Estado:** posição do carrinho e ângulo do poste com a vertical.
- ▶ **Ações:** mover o carrinho Δx para esquerda ou direita.
- ▶ **Recompensa:** +1 enquanto o poste estiver de pé; 0 caso contrário.

Exemplos: Xadrez



- ▶ **Estado:** configuração do tabuleiro.
- ▶ **Ação:** movimentar as peças.
- ▶ **Transição:** o próximo estado depende do **oponente** — não-determinística, e o adversário é **não-estacionário**.
- ▶ **Recompensa:** +1 ao vencer, 0 ao empatar, -1 ao perder — **só no fim** da partida.

Crédito futuro

A recompensa só chega no final, após **dezenas de lances**. A quais deles atribuir o mérito da vitória (ou a culpa da derrota)?

Dificuldades

O que torna o RL difícil — e cada exemplo já evidencia:

- ▶ **Estocasticidade:** estados, ações e recompensas são, em geral, aleatórios — o lance do oponente no *Xadrez* torna o próximo estado imprevisível.
- ▶ **Não-estacionariedade:** as distribuições mudam ao longo do tempo — o **adversário** no *Xadrez* adapta sua estratégia.
- ▶ **Crédito futuro** (*credit assignment*): a recompensa pode vir só no fim — no *Lunar Lander*, vale gastar combustível agora para garantir o pouso (+100 só no final); no *Cart Pole*, a queda do poste é consequência de ações tomadas vários passos antes.
- ▶ **Não-diferenciável:** não há gradiente da recompensa em relação aos parâmetros — não dá para treinar por *backprop* como no aprendizado supervisionado.

Multi-armed Bandits

Multi-armed Bandits

Imagine m caça-níqueis (*one-armed bandits*), ou um único com m alavancas. A cada jogada escolhemos um braço a e recebemos uma recompensa estocástica.

- ▶ Cada braço a tem um **valor** esperado **desconhecido**

$$q^*(a) = \mathbb{E}[r \mid a].$$

- ▶ Objetivo: maximizar a recompensa total ao longo de muitas jogadas.

É um problema de RL **simplificado**: um **único estado**, recompensa **imediate** e sem transições. O foco fica no dilema central de RL: **exploração** vs. **exploração**.

Estimando valores de ação

Não conhecemos $q^*(a)$, então o estimamos pela **média das recompensas** já observadas no braço a :

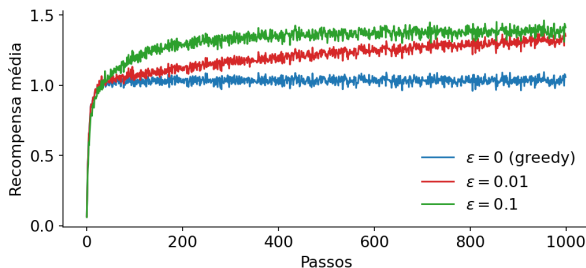
$$Q_t(a) = \frac{\text{soma das recompensas obtidas com } a}{\text{n}^{\text{o}} \text{ de vezes que } a \text{ foi escolhido}}$$

- ▶ Ação **gulosa** (*greedy*): $a_t = \arg \max_a Q_t(a)$.
- ▶ Agir sempre de forma gulosa (**exploração**) pode travar em um braço que *parece* bom e nunca descobrir o melhor.
- ▶ Precisamos, de vez em quando, **explorar** braços não-gulosos.

ϵ -greedy

A estratégia ϵ -greedy equilibra os dois objetivos:

- ▶ com probabilidade $1 - \epsilon$: escolhe o braço guloso (**exploração**);
- ▶ com probabilidade ϵ : escolhe um braço aleatório (**exploração**).



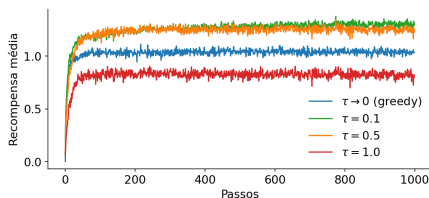
No *testbed* de 10 braços ($q^*(a) \sim \mathcal{N}(0, 1)$, $r \sim \mathcal{N}(q^*(a), 1)$, média de 2000 tarefas): o guloso estagna; explorar um pouco encontra braços melhores.

Seleção por softmax

Em vez de explorar de forma *uniforme*, podemos ponderar os braços pelo seu valor estimado, via **softmax** (seleção de Boltzmann):

$$\pi_t(a) = \frac{e^{Q_t(a)/\tau}}{\sum_b e^{Q_t(b)/\tau}}$$

onde τ é a **temperatura**: $\tau \rightarrow 0$ recupera a ação gulosa e τ grande aproxima da escolha uniforme.



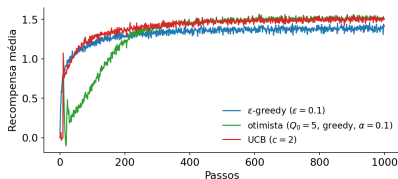
Uma temperatura **intermediária** supera tanto o guloso quanto a exploração excessiva. $\pi_t(a)$ já é uma **política** — voltaremos a ela.

Exploração dirigida: otimismo e UCB

Em vez de explorar por *sorteio*, podemos explorar de forma **dirigida**:

- ▶ **Valores iniciais otimistas**: inicializar $Q_0(a)$ acima de qualquer recompensa plausível (ex. $Q_0 = 5$): todo braço “decepciona”, e o guloso é forçado a experimentar todos.
- ▶ **UCB** (*Upper Confidence Bound*): bônus de incerteza para braços pouco visitados ($N_t(a) = n^o$ de escolhas de a):

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$



Implementação incremental

Guardar todas as recompensas para calcular a média é desnecessário. Podemos atualizar Q de forma **incremental**: após a $(k+1)$ -ésima recompensa r_{k+1} de um braço,

$$Q_{k+1} = Q_k + \frac{1}{k+1} [r_{k+1} - Q_k].$$

Essa é uma instância de um padrão recorrente em RL:

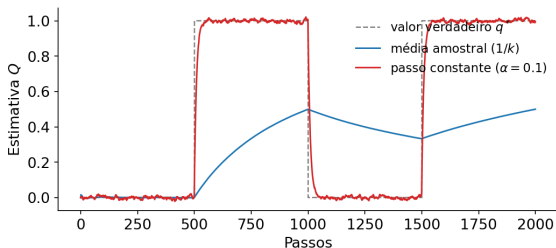
$$\text{Estimativa} \leftarrow \text{Estimativa} + \text{Passo} \cdot [\text{Alvo} - \text{Estimativa}].$$

Problemas não-estacionários

Se as distribuições de recompensa **mudam** ao longo do tempo — a **não-estacionariedade** vista na introdução —, a média amostral (peso $1/k$) responde cada vez mais devagar. Usamos um **passo constante** α :

$$Q_{k+1} = Q_k + \alpha [r_{k+1} - Q_k], \quad 0 < \alpha \leq 1.$$

Isso dá peso **exponencialmente decrescente** às recompensas antigas, permitindo *rastrear* mudanças.



Dos *bandits* ao problema completo

O que os *bandits* **não** têm:

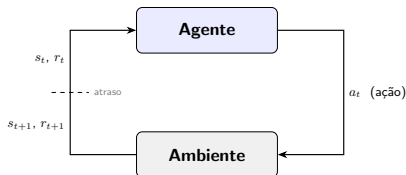
- ▶ **Estados:** toda jogada é idêntica — não há situação a observar antes de agir.
- ▶ **Transições:** a ação afeta só a recompensa imediata, nunca o futuro.
- ▶ **Crédito futuro:** sem consequências de longo prazo, não há mérito a distribuir.

A seguir devolvemos **estados** e **transições** ao problema. E levamos dos *bandits*:

- ▶ as estimativas de valor de ação (Q);
- ▶ as estratégias de exploração (ϵ -greedy, softmax, otimismo);
- ▶ o padrão Estimativa $\leftarrow$ Estimativa + Passo $\cdot$ [Alvo - Estimativa].

Formalização

Definições



$$s_0 \xrightarrow{a_0} (r_1, s_1) \xrightarrow{a_1} \cdots (r_t, s_t) \xrightarrow{a_t} (r_{t+1}, s_{t+1}) \xrightarrow{a_{t+1}} \cdots$$

Onde:

- ▶ $s_t \in \mathcal{S} = \{s_1, s_2, \dots\}$ é o estado do ambiente no tempo t , e $\mathcal{S}$ é o espaço de estados;
- ▶ $a_t \in \mathcal{A}(s_t)$ é a ação tomada pelo agente no tempo t , e $\mathcal{A}(s_t)$ é o espaço de ações disponíveis no estado s_t ;
- ▶ $r_t \in \mathbb{R}$ é a recompensa recebida no tempo t (após a ação a_{t-1} !).

A sequência acima é um *episódio* de interação do agente com o ambiente. O episódio é finito se acaba em $t = T$.

Definições: política

Ao longo de um episódio o agente utiliza a política (*policy*) π_t para escolher ações.

- ▶ $\pi_t(a \mid s)$ é a política do agente no tempo t , que retorna uma distribuição de probabilidade sobre as ações $a \in \mathcal{A}(s)$.
- ▶ O agente amostra sua política π_t ao escolher ações $a_t \sim \pi_t(a \mid s_t)$.
- ▶ A política é determinística se $\pi_t(a \mid s) = 1$ para uma ação a e 0 para todas as outras ações.
- ▶ A política é estacionária se não muda ao longo do tempo, ou seja, $\pi_t = \pi$ para todo t .

Após a ação a_t tomada pelo agente no estado s_t , o ambiente retorna:

- ▶ O próximo estado $s_{t+1} \sim P(s \mid s_t, a_t)$.
- ▶ A recompensa $r_{t+1} \sim P(r \mid s_t, a_t)$.

Objetivo do Agente

O objetivo do agente é maximizar a recompensa total recebida ao longo do episódio, chamada de **retorno** G .

- ▶ O retorno é dado por:

$$G = r_1 + r_2 + \dots + r_T + \dots$$

- ▶ O agente pode também considerar um retorno futuro descontado:

$$G = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots + \gamma^{T-1} r_T + \dots$$

onde γ é o fator de desconto, $0 \leq \gamma \leq 1$.

- ▶ Definimos $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ como o retorno a partir do tempo t .

Propriedade de Markov

Assumimos que todo problema em Reinforcement Learning satisfaz a *Propriedade de Markov*: o próximo estado s_{t+1} e a recompensa r_{t+1} dependem apenas do estado atual s_t e da ação tomada a_t .

Ou ainda:

$$P(s_{t+1} = s', r_{t+1} = r' \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = \\ P(s_{t+1} = s', r_{t+1} = r' \mid s_t, a_t)$$

Exemplos:

- ▶ Um tabuleiro de damas ou xadrez
- ▶ Movimento balístico de uma bala de canhão
- ▶ Um jogo de Pong

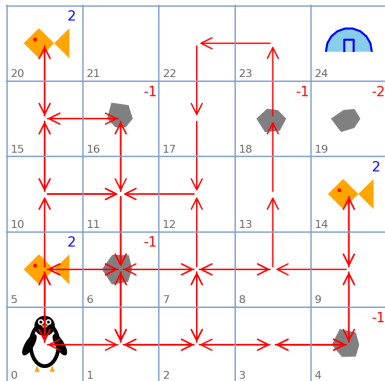
Probabilidades de transição e recompensa são dadas por:

$$\mathcal{P}_{ss'}^a = P(s_{t+1} = s' \mid s_t = s, a_t = a) \\ \mathcal{R}_{ss'}^a = \mathbb{E}[r_{t+1} \mid s_t = s, a_t = a]$$

Exemplo: o *gridworld* do pinguim

- ▶ **Estados:** as 25 células (o pinguim começa em 0).
- ▶ **Ações:** mover-se para cima, baixo, esquerda ou direita.
- ▶ **Recompensas:** peixes (+) e buracos (-).
- ▶ **Estado terminal:** o iglu (célula 24).

Com movimento **puramente aleatório**, o ambiente é um **processo de Markov**: o próximo estado depende apenas do atual.



Funções de Valor e Equações de Bellman

State Value Function e Action Value Function

- ▶ A **State Value Function** $V^\pi(s)$ é a expectativa do retorno a partir do estado s , segundo a política π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

- ▶ A **Action Value Function** $Q^\pi(s, a)$ é a expectativa do retorno a partir do estado s e ação a , segundo a política π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Podemos observar algumas relações entre as funções de valor:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$$
$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^\pi(s'))$$

Obs.: o $q^*(a)$ dos *bandits* é exatamente uma Action Value Function em um problema com um único estado.

Equações de Bellman

Usando as equações anteriores (ou partindo da definição de $V^\pi(s)$ e $Q^\pi(s, a)$), podemos escrever as **Equações de Bellman** para as funções de valor:

$$V^\pi(s) = \sum_a \pi(a | s) \left(\sum_{s'} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^\pi(s')) \right)$$
$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}_{ss'}^a \left(\mathcal{R}_{ss'}^a + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right)$$

- ▶ O valor de s é definido **recursivamente** pelo valor dos sucessores s' : recompensas distantes retropropagam de estado em estado.
- ▶ Essa recursão responde ao **crédito futuro** da introdução: o mérito de uma recompensa lá no fim chega a cada estado (e ação) através de V^π e Q^π .

Otimidade de π , V e Q I

Definimos uma ordem parcial para as políticas π :

$$\pi' \preceq \pi \Leftrightarrow V^{\pi'}(s) \leq V^{\pi}(s) \quad \forall s \in \mathcal{S}$$

Afirmção: Sempre existe uma política π^* que é maior ou igual a qualquer outra política π .

π^* é a política ótima e satisfaz

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad \forall s \in \mathcal{S}$$

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

$V^*(s)$ e $Q^*(s, a)$ satisfazem as seguintes relações:

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^*(s'))$$

Otimidade de π , V e Q II

E portanto, podemos escrever as Equações de *Otimidade* de Bellman para $V^*(s)$ e $Q^*(s, a)$:

$$V^*(s) = \max_a \left(\sum_{s'} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^*(s')) \right)$$
$$Q^*(s, a) = \sum_{s'} \mathcal{P}_{ss'}^a \left(\mathcal{R}_{ss'}^a + \gamma \max_{a'} Q^*(s', a') \right)$$

Por outro lado, se conhecemos $Q^*(s, a)$, podemos obter a política ótima π^* :

$$\pi^*(a | s) = \begin{cases} 1 & \text{se } a = \arg \max_{a'} Q^*(s, a') \\ 0 & \text{caso contrário} \end{cases}$$

Também podemos obter π^* a partir de $V^*(s)$:

$$\pi^*(a | s) = \begin{cases} 1 & \text{se } a = \arg \max_{a'} \sum_{s'} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^*(s')) \\ 0 & \text{caso contrário} \end{cases}$$

Métodos Elementares

Métodos Elementares: Programação Dinâmica

Métodos de Programação Dinâmica requerem conhecimento pleno de todas as probabilidades de transição $\mathcal{P}_{ss'}^a$ e expectativas de recompensa $\mathcal{R}_{ss'}^a$.

A ideia central: transformar as **Equações de Bellman** em regras de **atualização** — a equação de V^π vira uma *atribuição*, aplicada repetidamente até o ponto fixo.

São compostos por duas etapas, iteradas até a convergência:

- ▶ Policy Evaluation: Avalia a política atual π e calcula $V^\pi(s)$ ou $Q^\pi(s, a)$.
- ▶ Policy Improvement: Melhora a política atual π com base em $V^\pi(s)$ ou $Q^\pi(s, a)$.

Métodos Elementares: Programação Dinâmica

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s', r \mid s, a) [r + \gamma V(s')]$

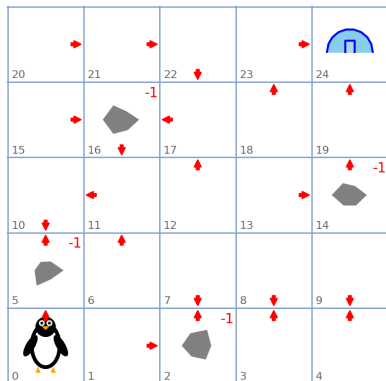
If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Programação Dinâmica: exemplo

No *gridworld*, partimos de uma política **aleatória** e alternamos *policy evaluation* e *policy improvement* até convergir.

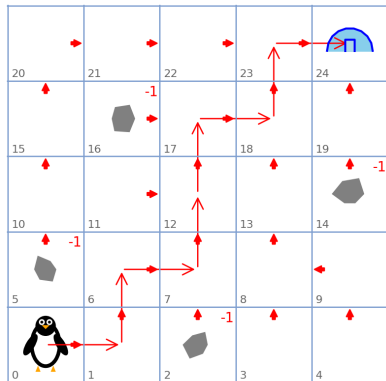
- **Política inicial** (aleatória): as setas apontam em direções arbitrárias.



Programação Dinâmica: exemplo

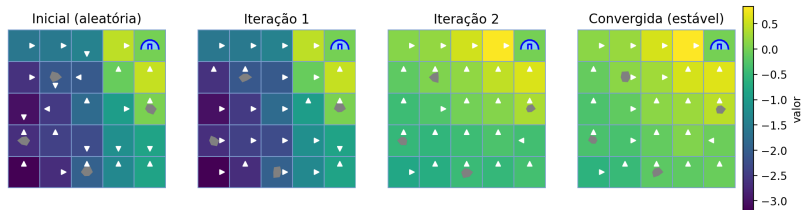
No *gridworld*, partimos de uma política **aleatória** e alternamos *policy evaluation* e *policy improvement* até convergir.

- **Política ótima:** as setas guiam o pinguim até o iglu, coletando peixes e evitando buracos.



Programação Dinâmica: convergência

A cada iteração, *policy evaluation* propaga o valor $V(s)$ (cor de fundo) e *policy improvement* reorienta as setas, até a política estabilizar.



Monte Carlo Methods

Métodos de Monte Carlo não requerem conhecimento pleno de todas as probabilidades de transição $\mathcal{P}_{ss'}^a$ e expectativas de recompensa $\mathcal{R}_{ss'}^a$.

- ▶ Utilizam amostras de episódios completos para estimar $V^\pi(s)$ ou $Q^\pi(s, a)$.
- ▶ A política é atualizada com base nas recompensas observadas.
- ▶ Não requerem conhecimento do modelo do ambiente.

Monte Carlo Methods

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

- $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
- $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
- $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

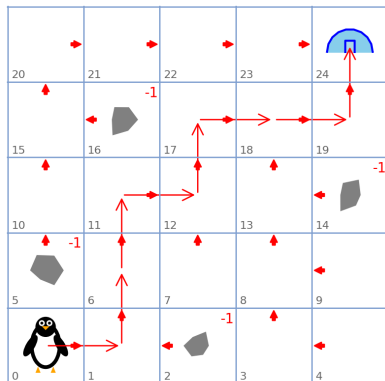
Loop forever (for each episode):

- Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
- Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- Loop for each step of episode, $t = T - 1, T - 2, \dots, 0$:
 - $G \leftarrow \gamma G + R_{t+1}$
 - Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:
 - Append G to $Returns(S_t, A_t)$
 - $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
 - $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$

Monte Carlo: exemplo

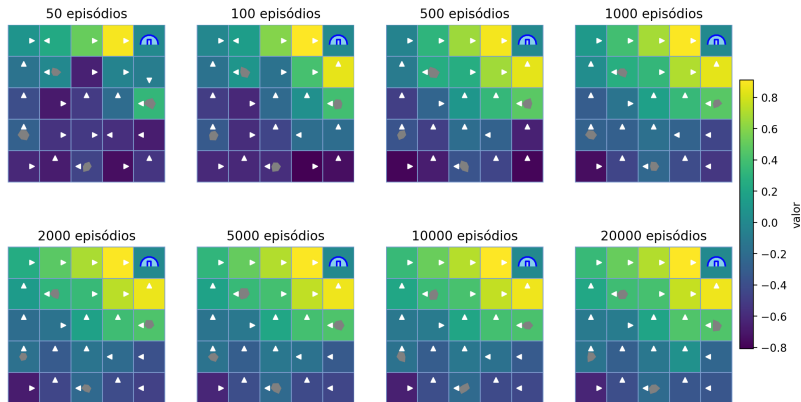
Sem conhecer o modelo, estimamos $Q(s, a)$ amostrando **episódios completos** (Monte Carlo com *exploring starts*) e agimos de forma gulosa em relação a Q .

- ▶ Mesmo *gridworld* da Programação Dinâmica.
- ▶ A política encontrada também leva o pinguim ao iglu, evitando os buracos — **sem usar** $\mathcal{P}_{ss'}^a$ nem $\mathcal{R}_{ss'}^a$.



Monte Carlo: convergência

Sem modelo, $V(s) = \max_a Q(s, a)$ (cor de fundo) e a política emergem da amostragem: com mais episódios, as estimativas se refinam e as setas convergem para o caminho do iglu.



Monte Carlo: limitações

Monte Carlo aprende sem modelo, mas o retorno G_t só fica disponível no **fim do episódio**:

- ▶ Em episódios longos (ou tarefas **contínuas**, que nunca terminam), a atualização demora — ou nunca acontece.
- ▶ G_t acumula a aleatoriedade de **todas** as ações e transições até o fim do episódio: o alvo tem **variância alta**.

A Programação Dinâmica não tinha esse problema: atualizava $V(s)$ a partir das estimativas dos vizinhos (*bootstrapping*), sem esperar episódio nenhum — mas exigia o **modelo**.

Pergunta: dá para atualizar a cada passo, como a Programação Dinâmica, mas aprendendo só da experiência, como Monte Carlo?

Temporal Difference Learning

Métodos de **Temporal Difference** (TD) respondem que sim, combinando ideias de Monte Carlo e de Programação Dinâmica:

- ▶ Como Monte Carlo, aprendem diretamente da experiência, **sem** conhecer o modelo do ambiente ($\mathcal{P}_{ss'}^a, \mathcal{R}_{ss'}^a$).
- ▶ Como Programação Dinâmica, atualizam estimativas a partir de outras estimativas (*bootstrapping*), **sem** esperar o fim do episódio.

Enquanto Monte Carlo usa o retorno observado G_t como alvo, TD usa o alvo **estimado** $r_{t+1} + \gamma V(s_{t+1})$:

$$\text{MC: } V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)]$$

$$\text{TD(0): } V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

onde α é a taxa de aprendizado. O “(0)” indica que o alvo usa **um único passo** antes do *bootstrap*: com n passos (ou a média ponderada deles, o TD(λ)), interpola-se entre TD(0) e Monte Carlo — voltaremos a essa ideia.

Ambas são instâncias do padrão visto nos *bandits* —

Estimativa $\leftarrow$ Estimativa + Passo $\cdot$ [Alvo – Estimativa] — mudando apenas o **alvo**.

Temporal Difference Learning

O termo entre colchetes é o **erro TD** (*TD error*):

$$\delta_t = \underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{alvo TD}} - V(s_t)$$

- ▶ Mede a diferença entre a estimativa atual $V(s_t)$ e uma estimativa melhor após observar r_{t+1} e s_{t+1} .
- ▶ Cada passo usa apenas a transição $(s_t, a_t, r_{t+1}, s_{t+1})$: aprendizado **online** e **incremental**.

É uma troca de **viés por variância** em relação a Monte Carlo:

- ▶ O alvo TD é **enviesado** — usa a própria estimativa $V(s_{t+1})$, que pode estar errada;
- ▶ mas tem **variância muito menor** que G_t : depende de **uma** transição, não do episódio inteiro. Na prática, TD costuma convergir mais rápido.

Controle: ε -greedy

O TD(0) acima resolve a **previsão**: avalia o V de uma política *fixa*. Para **controle** (encontrar uma boa política), repetimos o padrão da Iteração de Política e do Monte Carlo ES: estimamos $Q(s, a)$ e **melhoramos** a política agindo de forma gulosa em relação a ela.

Mas precisamos **explorar** para descobrir boas ações. Como nos *bandits*, a política ε -greedy equilibra exploração e exploração — agora por estado:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{se } a = \arg \max_{a'} Q(s, a') \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{caso contrário} \end{cases}$$

- ▶ Com probabilidade $1 - \varepsilon$: escolhe a melhor ação conhecida (*exploração*).
- ▶ Com probabilidade ε : escolhe uma ação aleatória (*exploração*).

Isso cria uma **tensão**: queremos aprender a política **ótima** (gulosa), mas precisamos nos **comportar** de forma exploratória. Há duas maneiras de resolvê-la.

On-policy vs. off-policy

Em todo método de controle há duas políticas em jogo:

- ▶ a **política de comportamento**: a que escolhe as ações e gera a experiência;
- ▶ a **política alvo**: a que estamos de fato avaliando e melhorando.

Definição

On-policy: a política alvo **é a mesma** que gera as ações.

Off-policy: aprende sobre uma política **diferente** da que gera as ações.

As duas saídas para a tensão do slide anterior:

- ▶ **On-policy** aceita o compromisso: aprende a melhor política *que ainda explora* (ex.: ϵ -greedy) — é o caso do **SARSA**.
- ▶ **Off-policy** separa os papéis: comporta-se com ϵ -greedy, mas aprende a política **gulosa** — é o caso do **Q-Learning**.

SARSA (on-policy)

Para controle, queremos aplicar o TD a Q , não a V . Trocando $V(s_{t+1})$ por $Q(s_{t+1}, a_{t+1})$ no alvo, o erro TD passa a precisar também da **próxima ação** — a quintupla $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ dá o nome ao **SARSA**:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

- ▶ A ação a_{t+1} usada no alvo é a **efetivamente tomada** pela política ϵ -greedy.
- ▶ É um método **on-policy**: avalia e melhora a mesma política que gera as ações.
- ▶ Aprende uma política que leva em conta a própria exploração (mais “cautelosa”).

SARSA (on-policy)

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R , S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'$; $A \leftarrow A'$

 until S is terminal

Q-Learning (off-policy)

Enquanto houver exploração, o SARSA aprende o Q da política ε -greedy — **não** o Q^* da política ótima. Dá para aprender Q^* diretamente, **sem parar de explorar**?

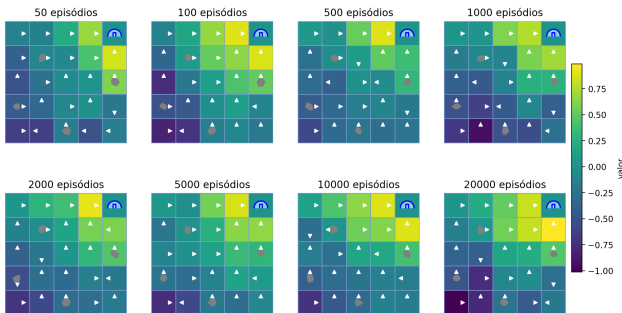
O **Q-Learning** responde que sim: usa no alvo a **melhor** ação possível em s_{t+1} , e não a ação efetivamente tomada:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

- ▶ É um método **off-policy**: a política alvo é a **gulosa** (o $\max_{a'}$), enquanto o comportamento segue explorando com ε -greedy.
- ▶ Compare com a equação de otimalidade de Bellman para Q^* : o $\max_{a'}$ aparece no lugar do $Q(s_{t+1}, a_{t+1})$ do SARSA.
- ▶ Converge para Q^* desde que todos os pares (s, a) continuem sendo visitados.

Q-Learning: convergência

Mesmo *gridworld*, agora com **Q-Learning** (ϵ -greedy, $\alpha = 0,2$): Q é atualizado a cada **transição**, sem modelo e sem esperar o fim do episódio.



Sem *exploring starts*, o valor se propaga ao longo do corredor efetivamente percorrido; estados raramente visitados quase não são atualizados.

Métodos Elementares: comparação

	Modelo?	Bootstrapping?	Episódio completo?	Política
Prog. Dinâmica	sim	sim	não	—
Monte Carlo ES	não	não	sim	on-policy
TD(0) (previsão)	não	sim	não	—
SARSA	não	sim	não	on-policy
Q-Learning	não	sim	não	off-policy

- ▶ Programação Dinâmica exige o modelo ($\mathcal{P}_{ss'}^a, \mathcal{R}_{ss'}^a$); os demais aprendem da **experiência**.
- ▶ Monte Carlo espera o episódio terminar; os métodos TD aprendem a cada **transição**.
- ▶ Todos chegam à **mesma política ótima** no *gridworld* — por caminhos diferentes.

Deep Reinforcement Learning

Deep Reinforcement Learning

Métodos tabulares guardam um valor por par (s, a) — inviável quando o espaço de estados é enorme ou contínuo (ex.: pixels de um jogo de Atari).

Ideia: aproximar Q por uma rede neural $Q(s, a; \theta)$ com parâmetros θ .

- ▶ A rede recebe o estado s e produz $Q(s, a; \theta)$ para cada ação.
- ▶ Treinamos minimizando o erro quadrático em relação ao alvo de Q-Learning:

$$L(\theta) = \mathbb{E} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta)}_{\text{alvo}} - Q(s, a; \theta) \right)^2 \right]$$

Duas fontes de **instabilidade**: o alvo depende da *própria* rede (alvo móvel) e as transições consecutivas são **correlacionadas**. Veremos como a **Deep Q-Network (DQN)** lida com cada uma.

Estudo de caso: jogando Atari com DQN

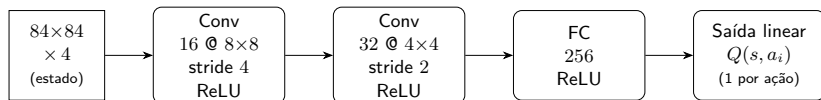


Um **único** agente, com a **mesma** arquitetura e hiperparâmetros, aprende a jogar 7 jogos do *Arcade Learning Environment*:

- ▶ Entrada: apenas os **pixels brutos** (210×160 RGB a 60 Hz), a recompensa (variação do placar) e o sinal de fim — como um jogador humano.
- ▶ Sem informação específica do jogo nem features projetadas à mão.
- ▶ Tela atual é **parcialmente observável** $\Rightarrow$ o estado empilha os últimos quadros.

Pré-processamento e arquitetura da rede

Pré-processamento ϕ : 210×160 RGB $\rightarrow$ tons de cinza $\rightarrow$ reescala $110 \times 84 \rightarrow$ recorte 84×84 ; empilha os **últimos 4 quadros** $\Rightarrow$ entrada $84 \times 84 \times 4$.



Há **uma saída por ação** (4 a 18 ações): um único *forward pass* calcula $Q(s, a)$ de todas as ações — em vez de passar a ação como entrada e repetir o cálculo.

Deep Q-learning with Experience Replay

Algorithm 1: Deep Q-learning with Experience Replay

```
Initialize replay memory  $D$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights
for episode = 1,  $M$  do
  Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
  for  $t = 1, T$  do
    With probability  $\varepsilon$  select a random action  $a_t$ 
    otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
    Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$ 
  end for
end for
```

Nota: “Deep Q-learning” é o *algoritmo*; **DQN** (*Deep Q-Network*) é a *rede* $Q(\cdot; \theta)$ que ele treina.

Na seleção da ação, o paper escreve $\max_a$, mas trata-se de $\arg \max_a$ (typo do original).

DQN: a função de valor aprendida



No *Seaquest*, o valor previsto **sobe** quando um inimigo aparece (A), atinge o **pico** quando o torpedo está prestes a acertá-lo (B) e **cai** ao valor original depois que ele some (C).

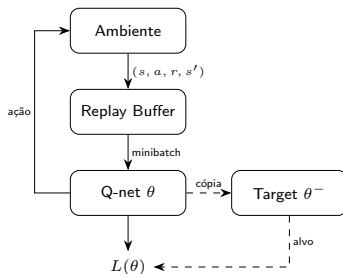
DQN na *Nature* (2015): Target Network

Segundo problema: o **alvo móvel** — o alvo muda a cada atualização de θ .

Target Network: uma cópia “congelada” θ^- dos parâmetros calcula o alvo, e só é atualizada periodicamente ($\theta^- \leftarrow \theta$):

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

- ▶ Entre atualizações de θ^- , o alvo é **fixo**: cada passo vira uma regressão comum.
- ▶ Com uma rede mais profunda (3 convoluções, FC de 512), atingiu desempenho de **nível humano** em dezenas de jogos.



Policy Gradient

Policy Gradient: motivação

Métodos baseados em valor (Q -learning, DQN) têm limitações:

- ▶ Difíceis de aplicar em espaços de ação **contínuos** (o $\max_a$ vira um problema de otimização) — ex.: o *Lunar Lander* da introdução, na versão em que o empuxo dos motores é um número real.
- ▶ A política ótima derivada de Q é sempre determinística — mas às vezes a política ótima é **estocástica**.

Ideia: parametrizar e otimizar a política **diretamente**, $\pi_\theta(a \mid s)$, maximizando o retorno esperado:

$$J(\theta) = \mathbb{E}_{\pi_\theta} [G_0]$$

Dedução: o truque da log-derivada

Uma **trajetória** $\tau = (s_0, a_0, r_1, s_1, \dots, s_T)$ alterna escolhas da **política** e respostas do **ambiente**:

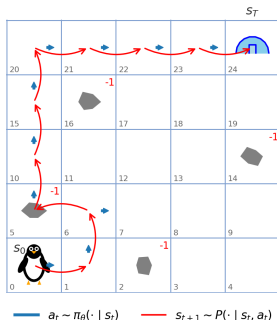
$$P(\tau; \theta) = P(s_0) \prod_t \pi_\theta(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

com retorno $G(\tau) = \sum_t \gamma^t r_{t+1}$ e objetivo $J(\theta) = \mathbb{E}_\tau [G(\tau)]$.

Usando $\nabla_\theta P = P \nabla_\theta \log P$ (**truque da log-derivada**):

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int P(\tau; \theta) G(\tau) d\tau = \int P(\tau; \theta) \nabla_\theta \log P(\tau; \theta) G(\tau) d\tau \\ &= \mathbb{E}_\tau [\nabla_\theta \log P(\tau; \theta) G(\tau)] \end{aligned}$$

O gradiente vira uma **esperança sob a própria política**: podemos estimá-lo por **Monte Carlo**, amostrando trajetórias.



Dedução: a dinâmica do ambiente desaparece

No log, o produto vira soma — e só a política depende de θ :

$$\nabla_{\theta} \log P(\tau; \theta) = \underbrace{\nabla_{\theta} \log P(s_0)}_{=0} + \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) + \underbrace{\sum_t \nabla_{\theta} \log P(s_{t+1} | s_t, a_t)}_{=0}$$

Substituindo:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(\tau) \right]$$

- ▶ As probabilidades de transição somem do gradiente: **não precisamos do modelo** do ambiente — só de **amostras** dele.
- ▶ **Causalidade**: ações em t não afetam recompensas passadas, então $G(\tau)$ pode ser trocado por G_t ; tomando a esperança condicional ($\mathbb{E}[G_t | s_t, a_t] = Q^{\pi_{\theta}}(s_t, a_t)$), recuperamos o enunciado do teorema.

REINFORCE

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a \mid s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot \mid \cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

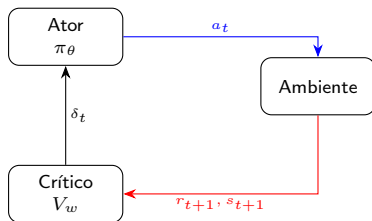
$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t \mid S_t, \theta)$$

Actor-Critic

Usa um **crítico** — uma segunda rede $V(s; w)$, com parâmetros próprios w , separados dos θ do ator — para estimar a **vantagem** (lembre: $A^\pi = Q^\pi - V^\pi$): o próprio **erro TD** é uma estimativa (de um passo) de $A^\pi(s_t, a_t)$:

$$\hat{A}_t = r_{t+1} + \gamma V(s_{t+1}; w) - V(s_t; w)$$

- ▶ **Ator** π_θ : escolhe as ações.
- ▶ **Crítico** V_w : avalia o quão boas elas foram, via erro TD δ_t .
- ▶ δ_t atualiza tanto o ator quanto o crítico.
- ▶ Base de algoritmos modernos como A2C/A3C, PPO e DDPG.



Actor-Critic

One-step Actor–Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a \mid s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, w)$

Parameters: step sizes $\alpha^\theta > 0$, $\alpha^w > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $w \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Initialize S (first state of episode)

$I \leftarrow 1$

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot \mid S, \theta)$

 Take action A , observe S' , R

$\delta \leftarrow R + \gamma \hat{v}(S', w) - \hat{v}(S, w)$ (if S' is terminal, then $\hat{v}(S', w) \doteq 0$)

$w \leftarrow w + \alpha^w \delta \nabla \hat{v}(S, w)$

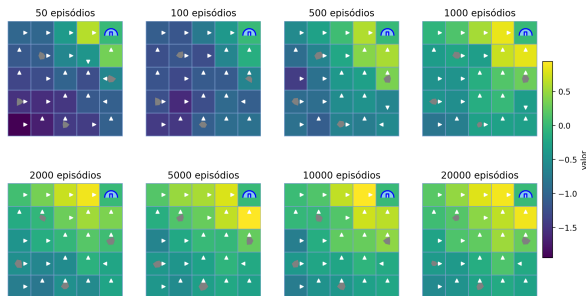
$\theta \leftarrow \theta + \alpha^\theta I \delta \nabla \ln \pi(A \mid S, \theta)$

$I \leftarrow \gamma I$

$S \leftarrow S'$

Actor-Critic: exemplo no gridworld

De volta ao *gridworld*: ator **softmax** sobre preferências $H(s, a)$ (a seleção de Boltzmann dos *bandits*, agora por estado) e crítico $V(s)$ tabular, ambos atualizados pelo erro TD δ_t a cada transição.



Cor de fundo: a estimativa $V(s)$ do **crítico**. Ao contrário do Q-Learning, ela avalia a política *atual* do ator — começa baixa (política quase aleatória) e cresce conforme o ator melhora. Setas: a ação mais provável do ator em cada estado.

A2C / A3C: Actor-Critic em escala

Duas melhorias práticas sobre o Actor-Critic de um passo:

- **Vantagem com n passos:** usa n recompensas reais antes do *bootstrap*:

$$\hat{A}_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n}; w) - V(s_t; w)$$

propaga recompensas mais rápido que 1 passo, com menos variância que Monte Carlo.

- **Ambientes paralelos:** vários atores coletam experiência ao mesmo tempo, **decorrelacionando** os dados *sem* replay buffer (que não serve para métodos on-policy).

A3C atualiza os parâmetros de forma *assíncrona*; **A2C** sincroniza os atores e atualiza em *batch* — mais simples e eficiente em GPU.

Na prática, porém, esses métodos são **sensíveis ao tamanho do passo**: uma atualização grande demais pode *degradar* a política de uma vez — a motivação dos próximos métodos.

Trust Region Policy Optimization (TRPO)

Por que: um passo grande demais pode destruir a política — e um passo pequeno no espaço de *parâmetros* pode ser enorme no espaço de *políticas*. Queremos o **maior passo possível** que ainda seja confiável.

Relembre o objetivo: $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$. Avaliá-lo em um candidato π_θ exigiria coletar **novas trajetórias** com ele — e os dados vieram de $\pi_{\theta_{\text{old}}}$. *Importance sampling* nas ações define o objetivo **substituto**:

$$L_{\theta_{\text{old}}}(\theta) = \mathbb{E}_t \left[\rho_t(\theta) \hat{A}_t \right], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

É uma boa aproximação **local** de J :

- ▶ em θ_{old} , **mesmo valor** e **mesmo gradiente** que J : perto, subir L é subir J ;
- ▶ longe, π_θ visitaria *outros* estados — mas os **estados** de L vêm de $\pi_{\theta_{\text{old}}}$: a aproximação deixa de valer.

Precisamos de uma noção de “**perto**” — no espaço de **políticas**.

Trust Region Policy Optimization (TRPO)

O TRPO maximiza o substituto dentro de uma **trust region**: a região, medida pela divergência KL, onde **confiamos** na aproximação $L_{\theta_{\text{old}}}$:

$$\max_{\theta} L_{\theta_{\text{old}}}(\theta) \quad \text{sujeito a} \quad \mathbb{E}_t[D_{\text{KL}}[\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}]] \leq \delta$$

- ▶ A restrição de divergência KL limita o passo **no espaço de políticas** (e não no de parâmetros), garantindo melhora (aproximadamente) **monotônica**.
- ▶ Custo: otimização de **segunda ordem** (gradiente natural, via gradientes conjugados + *line search*) — complexa de implementar e escalar.

PPO: objetivo clipado

Retome a **razão de probabilidades** do objetivo substituto do TRPO:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad \rho_t(\theta_{\text{old}}) = 1.$$

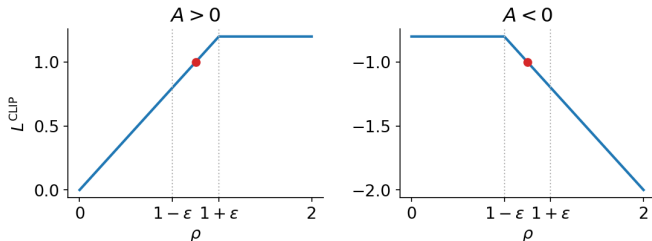
Maximizar o substituto $L_{\theta_{\text{old}}}(\theta) = \mathbb{E}[\rho_t(\theta) \hat{A}_t]$ **sem** a restrição de KL leva a passos grandes demais. O PPO usa o objetivo **clipado**:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

onde $\hat{A}_t$ é a vantagem estimada e $\varepsilon \approx 0,2$. O min com a versão clipada **remove o incentivo** de mover ρ_t para muito longe de 1.

Nota: o paper escreve $r_t(\theta)$; usamos ρ_t para não confundir com a recompensa r_t .

PPO: objetivo clipado



- ▶ A curva azul é L^{CLIP} ; o ponto vermelho marca $\rho_t(\theta_{\text{old}}) = 1$, de onde toda atualização parte.
- ▶ $\hat{A}_t > 0$ (ação boa): o ganho satura em $\rho_t = 1 + \epsilon$ — não compensa aumentar mais a probabilidade.
- ▶ $\hat{A}_t < 0$ (ação ruim): o objetivo satura em $\rho_t = 1 - \epsilon$ — limita o quanto a probabilidade é reduzida de uma vez.

PPO: objetivo completo e algoritmo

O objetivo combina o termo clipado com a perda do crítico e um bônus de **entropia** (incentiva exploração):

$$L(\theta) = \mathbb{E}_t \left[L^{\text{CLIP}}(\theta) - c_1 \underbrace{(V_\theta(s_t) - V_t^{\text{alvo}})^2}_{\text{perda do crítico}} + c_2 \underbrace{S[\pi_\theta](s_t)}_{\text{entropia}} \right]$$

A cada iteração:

1. Coleta trajetórias com a política atual $\pi_{\theta_{\text{old}}}$.
2. Estima as vantagens $\hat{A}_t$, tipicamente via **GAE** (média exponencial de erros TD: menos variância, um pouco de viés). [4]
3. Otimiza $L(\theta)$ por **várias épocas** em *minibatches* (reusa os dados).
4. Atualiza $\theta_{\text{old}} \leftarrow \theta$.

Estável, eficiente em amostras e simples: tornou-se o *default* em controle contínuo e a base do **RLHF** no alinhamento de LLMs.

[4] 2016 — John Schulman et al. — *High-Dimensional Continuous Control Using Generalized Advantage Estimation*

[5] 2017 — John Schulman et al. — *Proximal Policy Optimization Algorithms*

PPO vs. TRPO vs. A2C

Os três são métodos **on-policy** de gradiente da política com crítico. Diferem em *como* controlam o tamanho do passo da política:

	A2C	TRPO	PPO
Controle do passo	<i>learning rate</i>	restrição D_{KL}	<i>clip</i> de ρ_t
Ordem	primeira	segunda	primeira
Reuso dos dados	1 passo	1 passo	várias épocas
Eficiência amostral	baixa	média	alta
Complexidade	baixa	alta	baixa
Estabilidade	baixa	alta	alta

- ▶ **A2C**: gradiente com vantagem, simples, mas sensível ao passo.
- ▶ **TRPO**: melhora monotônica via *trust region*, ao custo de otimização de segunda ordem.
- ▶ **PPO**: aproxima o TRPO com um objetivo clipado de primeira ordem — melhor custo-benefício.

Resumo

- ▶ **Formalização:** agente e ambiente, Propriedade de Markov, retorno G_t , funções de valor V^π , Q^π e Equações de Bellman.
- ▶ **Com modelo:** Programação Dinâmica (*policy evaluation* + *policy improvement*).
- ▶ **Sem modelo:** Monte Carlo (episódios completos) e TD (*bootstrapping*, online) — SARSA e Q-Learning.
- ▶ **Aproximação de função:** DQN — *replay buffer* e *target network* levam o Q-Learning aos pixels do Atari.
- ▶ **Política direta:** REINFORCE → Actor-Critic/A2C → TRPO → PPO, o *default* em controle contínuo e a base do RLHF.

O que ficou de fora

Métodos *model-based*, *offline* RL, estratégias de exploração mais ricas (Thompson sampling, curiosidade), AlphaGo/MuZero e o RLHF em detalhe.

Referências I

- [1] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: *International Conference on Machine Learning (ICML)*. 2016, pp. 1928–1937.
- [2] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533.
- [3] Volodymyr Mnih et al. “Playing Atari with Deep Reinforcement Learning”. In: *arXiv preprint arXiv:1312.5602* (2013).
- [4] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations (ICLR)*. 2016.
- [5] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [6] John Schulman et al. “Trust Region Policy Optimization”. In: *International Conference on Machine Learning (ICML)*. 2015, pp. 1889–1897.

Referências II

- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: an introduction*. en. Second edition. Adaptive computation and machine learning series. Cambridge, Massachusetts: The MIT Press, 2018. ISBN: 978-0-262-03924-6.
- [8] Richard S. Sutton et al. “Policy gradient methods for reinforcement learning with function approximation”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2000, pp. 1057–1063.
- [9] Ronald J. Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine Learning* 8.3 (1992), pp. 229–256.